

Practica Final - Sistemas Operativos

Lautaro Bachmann

Contents

1	F1	5
1.1	Ej1 (CORREGIR)	5
1.1.1	Temas	5
1.1.2	HowTo	5
1.1.2.1	fork();	5
1.1.2.2	execvp();	5
1.1.2.3	FD:	5
1.1.2.4	close(FD);	5
1.1.2.5	open(archivo, flag);	6
1.1.2.6	read(FD, buffer, cantBytes);	6
1.1.2.7	write(FD, buffer, CantBytes);	6
1.1.2.8	argc	6
1.1.2.9	Pasos	6
1.1.3	Resolucion (CORREGIR)	7
1.1.4	Correcciones	8
1.2	Ej2 (DONE)	8
1.2.1	Temas	8
1.2.2	HowTo	9
1.2.2.1	Se representa	9
1.2.2.2	Pasar de virtual a física	9
1.2.2.3	Pasar física a virtual	9
1.2.2.4	Esquema (10, 22)	9
1.2.3	Resolucion	10
1.2.3.1	A) DONE	10
1.2.3.2	B)	11
1.3	Ej3	14
1.3.1	Temas	14

1.3.2	HowTo	14
1.3.2.1	Pasos A	14
1.3.2.2	Pasos B	14
1.3.2.3	Pasos C	14
1.3.3	Resolucion	15
1.3.4	Correccion	16
1.4	Ej4	16
1.4.1	Temas	16
1.4.2	HowTo	16
1.4.3	Resolucion	17
1.5	Ej5	17
1.5.1	Temas	17
1.5.2	HowTo	17
1.5.2.1	A)	17
1.5.2.2	B)	18
1.5.2.3	C)	18
1.5.3	Resolucion	19
1.5.4	Correcciones	19
1.6	Ej6 (CORREGIR)	20
1.6.1	Temas	20
1.6.2	HowTo	20
1.6.2.1	a)	20
1.6.2.2	b)	20
1.6.2.3	c)	20
1.6.3	Resolucion	21
1.6.3.1	a)	21
1.6.3.2	b)	22
1.6.4	Correcciones	22
2	F2	23
2.0.1	Ej1	23
2.0.2	Temas	23
2.0.3	HowTo	23
2.0.4	Resolucion	23
2.1	Ej2	23
2.1.1	Temas	24
2.1.2	HowTo	24
2.1.3	Resolucion	24
2.2	Ej3	24
2.2.1	Temas	25
2.2.2	HowTo	25
2.2.3	Resolucion	25
2.3	Ej4	26
2.3.1	Temas	26

	2.3.2	HowTo	26
	2.3.3	Resolucion	26
2.4	Ej5		27
	2.4.1	Temas	27
	2.4.2	HowTo	27
	2.4.3	Resolucion	28
		2.4.3.1 a)	28
		2.4.3.2 b)	28
	2.4.4	Correcciones	28
2.5	Ej6		29
	2.5.1	Temas	29
	2.5.2	HowTo	29
	2.5.3	Resolucion	31
		2.5.3.1 a) y b)	31
		2.5.3.2 c)	32
3	F3		32
3.1	Ej1		32
	3.1.1	Temas	32
	3.1.2	HowTo	32
	3.1.3	Resolucion	32
3.2	Ej2		32
	3.2.1	Temas	33
	3.2.2	HowTo	33
	3.2.3	Resolucion	33
	3.2.4	Correccion	34
3.3	Ej3		34
	3.3.1	Temas	35
	3.3.2	HowTo	35
	3.3.3	Resolucion	35
3.4	Ej4		35
	3.4.1	Temas	36
	3.4.2	HowTo	36
	3.4.3	Resolucion	36
3.5	Ej5		37
	3.5.1	Temas	38
	3.5.2	HowTo	38
	3.5.3	Resolucion	38
3.6	Ej6		38
	3.6.1	Temas	39
	3.6.2	HowTo	39
	3.6.3	Resolucion	41
3.7	Ej7		42
	3.7.1	Temas	44

3.7.2	HowTo	44
3.7.3	Resolucion	44
4	F4	45
4.1	Ejercicio 1:	45
4.2	Ejercicio 2	45
4.3	Ejercicio 3	45
4.4	Ejercicio 4	45
4.5	Ejercicio 5	45
5	F5 (final Eze)	45
5.1	Ej3	45
5.1.1	Howto	45
5.2	Ej4	46
5.2.1	Howto	46
5.3	Ej5	46
5.3.1	Howto	46
5.4	Ej tabla create read/writes	46
5.4.1	Howto	46
5.4.1.1	Creacion	46
5.4.1.2	Escritura	47
5.5	Ej tabla open read/writes (no tomado, pero lo anoto por las dudas) . . .	47
5.5.1	Howto	47
5.5.1.1	Apertura	47
5.5.1.2	Lectura	47
6	Final 7-12-2023	47
6.1	Ej6	47
6.1.1	Howto	47
6.1.1.1	Calcular tamaño i-bmap	47
6.1.1.2	Calcular tamaño d-bmap	48
6.1.1.3	Calcular tamaño inode-table	48
6.1.1.4	Calcular tamaño data-region	48
7	Extra	48
7.1	Caps final	48
7.2	Notas	48
7.2.1	¿Qué es máxima concurrencia?	49

1 F1

1.1 Ej1 (CORREGIR)

Ejercicio 1. Explique detalladamente como funcionan estos tres programas indicando cuantos procesos se crean, que hacen los padres, etc.

Suponga que en ambos casos se invocan con, y el archivo `entrada.txt` existe y no es ejecutable.

```
$ echo "Hola" > entrada.txt
$ ./a.out entrada.txt
```

<pre>int main(int argc, char ** argv) { close(6); close(4); close(2); close(0); open(argv[1], 0); open(argv[0], 0); open(argv[0], 0); open(argv[1], 0); read(2, buf, L); write(1, buf, L); }</pre>	<pre>int main(int argc, char ** argv) { execvp(argv[argc-1], argv); if (fork()) main(argc, argv); }</pre>	<pre>int main(int argc, char ** argv) { fork(); execvp(argv[0], argv); main(argc, argv); }</pre>
--	---	--

1.1.1 Temas

- Procesos
- Api procesos

1.1.2 HowTo

1.1.2.1 fork(); Crea un proceso. Devuelve 0 en caso hijo, >0 en caso padre. El if es para indicar que hace el padre y el hijo en base al return

1.1.2.2 execvp(); Esta funcion recibe como argumento un programa y un array, el array contiene el programa y argumentos.

IMPORTANTE: NO CREA UN NUEVO PROCESO, SI NO QUE REEMPLAZA AL QUE SE ESTABA EJECUTANDO

1.1.2.3 FD: Un file descriptor es un int que sirve para identificar archivos abiertos.

0 STDIN (input) 1 STDOUT (output) 2 STDERR(error) lo

1.1.2.4 close(FD); Cierra el archivo y FD que le pasamos, sin mas.

1.1.2.5 open(archivo, flag); Abre en el FD mas chico (numero) el archivo que le pasamos con la flag que le pasemos xdd. si la flag es 0, entonces **solo tiene permiso de lectura.**

1.1.2.6 read(FD, buffer, cantBytes); Lee el FD y lo mete en el buffer. La cantBytes vendria a ser el tamaño

1.1.2.7 write(FD, buffer, CantBytes); Escribe en el FD lo que haya en buffer, la cantBytes seria el tam del buffer

1.1.2.8 argc Incluye nombre del programa. Osea que si no se pasan argumentos argc es 1

1.1.2.9 Pasos

Para cada programa:

1. Describir brevemente que hace
2. Calcular cuantas veces se ejecuta `fork()`
3. Determinar que hace el padre
4. Determinar que hacen los hijos

1.1.3 Resolucion (CORREGIR)

Ejercicio 1. Explique detalladamente como funcionan estos tres programas indicando cuantos procesos se crean, que hacen los padres, etc.
Suponga que en ambos casos se invocan con, y el archivo `entrada.txt` existe y no es ejecutable.

```
$ echo "Hola" > entrada.txt  
$ ./a.out entrada.txt
```

4	B <pre>int main(int argc, char ** argv) { close(6); close(4); close(2); close(0); open(argv[1], 0); open(argv[0], 0); open(argv[0], 0); open(argv[1], 0); read(2, buf, L); write(1, buf, L); }</pre>	C <pre>int main(int argc, char ** argv) { fork(); execvp(argv[0], argv); main(argc, argv); }</pre>
---	--	--

Descripción General:

- A:** - Crea varios FD: entre ellos `STDIN` y `STDERR`
- Abre en modo lectura `entrada.txt` y `a.out` dos veces
- Lee `STDERR` y lo carga en `buf`, con tamaño `L`
- Imprime por `STDOUT` el contenido de `buf`, es decir, imprime el error obtenido previamente. También con tamaño `L`.
- B:** - El programa intenta ejecutar `entrada.txt`, pero como no es ejecutable falla y continúa la ejecución
- En el `if` el padre no hace nada (`PID=0`, entonces no entra al `if`) y el hijo ejecuta el mismo programa de manera recursiva.
- Una descripción más general del programa puede ser que es un bucle infinito
- C:** - El programa hace un `fork` y tanto el padre como el hijo ejecutan `execvp` con `a.out`, generando así un bucle infinito que en cada iteración crea dos procesos.
- No se llega a ejecutar nunca la tercer línea, pero si se hiciera el efecto sería el mismo
- Este programa es básicamente una bomba `fork`

Cantidad de procesos:

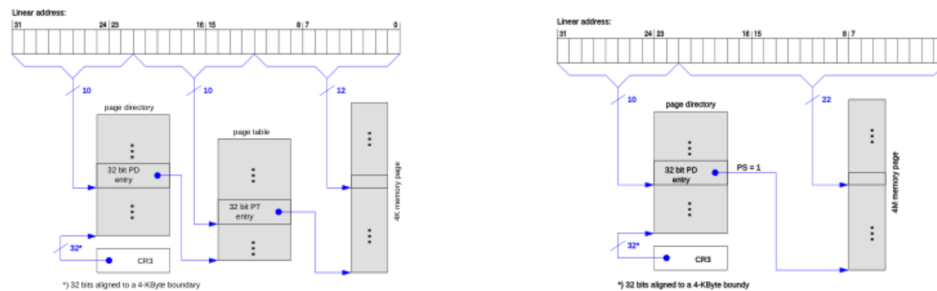
- A:** Solo hay un proceso
B: Se crea un proceso en cada "iteración". El padre sale mientras que el hijo sigue
C: Se crean procesos a un ritmo exponencial.

1.1.4 Correcciones

- Hablar de que FDs tiene cada archivo
- Tener en cuenta que si se cierra `STRERR` (2) otro archivo puede ocupar ese FD
- Tener en cuenta que el **hijo** es el que devuelve 0 en `fork()`

1.2 Ej2 (DONE)

Ejercicio 2. Para el sistema de paginado i386 con el bit de PSE (page size extension) dentro de `CR4` habilitado, el esquema de paginación puede ser (10,10,12) o (10,22), dependiendo del bit 7 de cada entrada del *pagedir* denominado PS (page size).



Sabiendo que `CR3=0x0BEBE` y que el contenido de los marcos físicos son los siguientes:

0x0BABA	0x0BEBE	0x0B0B0
-----	-----	-----
0x000: 0x10000 (P)	0x000: 0x0BEBE (P)	0x000: 0x00000 (P)
0x001: 0x10000 (P)	0x001: 0x103FF (P,PS)	0x001: 0x00001 (P)
...	...	...
...	...	...
0x3FE: 0x10000 (P)	0x3FE: 0x0B000 (P,PS)	0x3FE: 0x001FE (P)
0x3FF: 0x00000 (P)	0x3FF: 0x0BABA (P)	0x3FF: 0x001FF (P)

- (a) Pasar de virtual a física: 0x00000505, 0x00401FEE, 0x00402FEE, 0xFF8338FF.
 (b) Pasar de física a todas las virtuales: 0x00000000, 0x0BFFFFFF, 0x10000AAA.

1.2.1 Temas

- Address spaces
- Address translation
- Paging

1.2.2 HowTo

1.2.2.1 Se representa

(BITS_PAGE_DIR, BITS_PAGE_TABLE, BITS_OFFSET)

1.2.2.2 Pasar de virtual a física

1. Tomar primeros n bits especificados y ver a que entrada de la page directory corresponde
2. Ver a que page table apunta la entrada de la page directory
3. Tomar el número de entrada de la page table del segundo grupo de m bits especificados
4. Reemplazar los primeros 5 dígitos de la memoria virtual con la dirección del marco físico de la entrada
5. Revisar demás bits entrada para ver acceso y si está presente

1.2.2.3 Pasar física a virtual

Parecido al anterior, solo que hay que ver todos los caminos que llevan a esa dirección física.

1. Ver todas las entradas de las page tables que apuntan al frame (5 primeros bytes)
2. Determinar si estas tablas son accesibles desde la page dir
3. Determinar esquema a utilizar
4. Hallar entrada page dir, entrada page table y bits offset o solo entrada page dir y offset según corresponda.
5. Armar dirección concatenando los datos necesarios

1.2.2.4 Esquema (10, 22)

Para este esquema hay que tomar solo primeros 10 bits del frame number y luego concatenar con el offset

1.2.3 Resolución

1.2.3.1 A) DONE

Sabiendo que CR3=0x0BEBE y que el contenido de los marcos físicos son los siguientes:

0x0BABA	0x0BEBE	0x0B0B0
-----	-----	-----
0x000: 0x10000 (P)	0x000: 0x0BEBE (P)	0x000: 0x00000 (P)
0x001: 0x10000 (P)	0x001: 0x103FF (P,PS)	0x001: 0x00001 (P)
...	...	...
...	...	...
0x3FE: 0x10000 (P)	0x3FE: 0x0B000 (P,PS)	0x3FE: 0x001FE (P)
0x3FF: 0x00000 (P)	0x3FF: 0x0BABA (P)	0x3FF: 0x001FF (P)

(a) Pasar de virtual a física: 0x00000505, 0x00401FEE, 0x00402FEE, 0xFF8338FF.

(b) Pasar de física a todas las virtuales: 0x00000000, 0x0BFFFFFF, 0x10000AAA.

$$\begin{aligned}
 a) \quad & 0x \underbrace{0000}_{0} \underbrace{0505}_{0} = 0x0BEBE505 \\
 & 0x \underbrace{00401FEE}_{\substack{1 \\ PS}} = \overset{10b}{FrameNum} + \overset{22b}{offset} \\
 & = 0001\ 0000\ 0000\ 0001\ 1111\ 1110\ 1110 \\
 & = 0x\ 1\ 0\ 0\ 1\ F\ E\ E \\
 & = 0x1001FEE
 \end{aligned}$$

$$\begin{aligned}
 & 0x \underbrace{00402FEE}_{\substack{1 \\ PS}} = \overset{10b}{FrameNum} + \overset{22b}{offset} \\
 & = 0001\ 0000\ 0000\ 0001\ 1111\ 1110\ 1110 \\
 & = 0x\ 1\ 0\ 0\ 2\ F\ E\ E \\
 & = 0x1002FEE
 \end{aligned}$$

$$\begin{aligned}
 & 0x \underbrace{FF8338FF}_{\substack{\downarrow \\ 11\ 1111\ 1110}} = 0x\ 0B0 + 0x0338FF \\
 & = 0x0B0338FF \\
 & 0x\ 3\ F\ E
 \end{aligned}$$

1.2.3.2 B)

0x00000000
Frame Offset

Razonamiento

Lo primero que debemos hacer es buscar que entradas de las page tables apunten al frame.

En este caso son dos, 0x3FF en 0x0BABA y 0x000 en 0x0B0B0.

Como no tenemos ninguna entrada en 0x0BEBE que apunte a 0x0B0B0, nos centramos en la primera opción.

Armar direccion:

Entrada page dir = 0x3FF

Entrada page table = 0x3FF

Offset = 0x000

Teniendo en cuenta que no hay bit PS, tenemos lo sig.

Dir = 1111 1111 11 11 1111 1111 0000 0000 0000
= 0x F F F F F 0 0 0
= 0xFFFFF000

Sabiendo que CR3=0x0BEBE y que el contenido de los marcos físicos son los siguientes:

0x0BABA	0x0BEBE	0x0B0B0
-----	-----	-----
0x000: 0x10000 (P)	0x000: 0x0BEBE (P)	0x000: 0x00000 (P)
0x001: 0x10000 (P)	0x001: 0x103FF (P,PS)	0x001: 0x00001 (P)
...	...	...
...	...	...
0x3FE: 0x10000 (P)	0x3FE: 0x0B000 (P,PS)	0x3FE: 0x001FE (P)
0x3FF: 0x00000 (P)	0x3FF: 0x0BABA (P)	0x3FF: 0x001FF (P)

(a) Pasar de virtual a física: 0x00000505, 0x00401FEE, 0x00402FEE, 0xFF8338FF.

(b) Pasar de física a todas las virtuales: 0x00000000, 0x0BFFFFFF, 0x10000AAA.

0x0BFFFFFF

FN = 0x0BFFF

Como no hay ningún frame que coincida con los primeros 5 bytes para un esquema (10, 10, 12), probemos buscar con el esquema (10, 22)

FN = 0000 1011 11 = 00 0010 1111 = 0x02F

Como 0x02F tampoco coincide con ningún frame, concluimos que la dirección física no puede ser mapeada a ninguna dirección virtual

Sabiendo que CR3=0x0BEBE y que el contenido de los marcos físicos son los siguientes:

0x0BABA	0x0BEBE	0x0B0B0
-----	-----	-----
0x000: 0x10000 (P)	0x000: 0x0BEBE (P)	0x000: 0x00000 (P)
0x001: 0x10000 (P)	0x001: 0x103FF (P,PS)	0x001: 0x00001 (P)
...	...	...
...	...	...
0x3FE: 0x10000 (P)	0x3FE: 0x0B000 (P,PS)	0x3FE: 0x001FE (P)
0x3FF: 0x00000 (P)	0x3FF: 0x0BABA (P)	0x3FF: 0x001FF (P)

(a) Pasar de virtual a física: 0x00000505, 0x00401FEE, 0x00402FEE, 0xFF8338FF.

(b) Pasar de física a todas las virtuales: 0x00000000, 0x0BFFFFFF, 0x10000AAA.

0x10000AAA

FN = 0x10000

El frame puede ser accedido desde las siguientes entradas en 0x0BABA: 0x000, 0x001 y 0x3FE.

Como el único camino a 0x0BABA desde 0x0BEBE es a través de la entrada 0x3FF, tenemos lo siguiente:

entrada page dir = 0x3ff

entrada page table $\in \{0x000, 0x001, 0x3FE\}$

offset = 0xAAA

Armemos las direcciones:

0x3FF = 1111 1111 11

0x3FE = 11 1111 1110

EPT = 0x000 $\Rightarrow$ 1111 1111 1100 0000 0000 + 0xAAA
= 0xFFC00AAA

EPT = 0x001 $\Rightarrow$ 1111 1111 1100 0000 0001 + 0xAAA
= 0xFFC01AAA

EPT = 0x3FE $\Rightarrow$ 1111 1111 1111 1111 1110 + 0xAAA
= 0xFFFFEAAA

1.3 Ej3

Ejercicio 3. Considere el siguiente multiprograma de dos componentes P0 y P1, donde la **atomicidad es línea a línea** y todas las variables (i, j, a) son compartidas.

Pre: $i=0 \wedge j=0$	
P0 : while($i < 16$) { $a[i] = j$; $++i$; } 	P1 : while($j < 32$) { $++j$; }
Post: ?	

- Expresar de manera concisa todos resultados posibles en el arreglo $a[0, 16)$. Explique.
- ¿Se puede cumplir con la postcondición $(\forall k : 1 \leq k < 16 : a[k-1] < a[k])$? Explique.
- Sincronice con semáforos para lograr que **siempre** se satisfaga la postcondición $(\forall k : 1 \leq k < 16 : a[k-1] < a[k])$ sin hacer peligrar la terminación y maximizando la concurrencia. Solo puede agregar ifs para hacer los wait y post, no se puede tocar el resto del programa.

1.3.1 Temas

- Concurrency and threads
- Semaphores

1.3.2 HowTo

1.3.2.1 Pasos A Pensar en el mayor y menor resultado posible. Determinar números posibles dentro de ese intervalo.

Por ejemplo, si el menor número posible es 100 y el mayor número posible es 200. Y puedes armar números de la forma $2^n + r$, con $r \in \{0, 1\}$, sabes que puedes llegar a cualquier número entre 100 y 200

1.3.2.2 Pasos B Usar criterio similar al anterior

1.3.2.3 Pasos C

- Hacer un semaforo por cada programa.
- Hacer que se vayan ejecutando intercaladamente.

1.3.3 Resolucion

Ejercicio 3. Considere el siguiente multiprograma de dos componentes P0 y P1, donde la **atomicidad es línea a línea** y todas las variables (i, j, a) son compartidas.

Pre: $i=0 \wedge j=0$	
P0 : while($i < 16$) { $a[i] = j$; $++i$; } 	P1 : while($j < 32$) { $++j$; }
Post: ?	

P1
P0

- Expresar de manera concisa todos resultados posibles en el arreglo $a[0, 16)$. Explique.
- ¿Se puede cumplir con la postcondición ($\forall k : 1 \leq k < 16 : a[k-1] < a[k]$)? Explique.
- Sincronice con semáforos para lograr que **siempre** se satisfaga la postcondición ($\forall k : 1 \leq k < 16 : a[k-1] < a[k]$) sin hacer peligrar la terminación y maximizando la concurrencia. Solo puede agregar ifs para hacer los wait y post, no se puede tocar el resto del programa.

a)

veamos los casos border :

El valor mínimo que pueden tomar los elementos de a es 0.

Esto se logra ejecutando todo P0 y después todo P1

El valor máximo que pueden tomar los elementos de a es 32

Esto se logra ejecutando todo P1 y después todo P0

Por lo cual, tenemos a_i puede tomar valores en el rango de 0 a 32,

siempre que se cumple que $a_{i-1} \leq a_i$

Esto incluye todas las posibles combinaciones que cumplen con el predicado anterior.

Escribiéndolo de otra forma: ($\forall k : 0 \leq k \leq 32 : a[k-1] \leq a[k]$)

- Si se puede. Lo que hay que hacer es intercalar la ejecución de P1 y P0 (comenzando con P1) hasta que $i = 16$, para luego ejecutar el resto de P1

```

1 sem_init(&p0, 0, 0)
2 sem_init(&p1, 0, 1)
3
4 while(1<16) {           | while(j<32) {
5     sem_wait(&p0)       |     sem_wait(&p1)
6     a[i] = j;           |     ++j;
7     ++i;                |     sem_post(&p0)
8     sem_post(&p1)       | }
9 }
```

```

sem_init(&p0, 0, 0)
sem_init(&p1, 0, 1)

while(1<16) { | while(j<32) {
    sem_wait(&p0) | sem_wait(&p1)
    a[i] = j;    | ++j;
    ++i;         | sem_post(&p0)
    sem_post(&p1) | }
}
```

1.3.4 Correccion

Yo lo interprete como que tiene que ir de 1 a 16, pero creo que eso está mal

1.4 Ej4

Ejercicio 4. Debajo se muestra solución **incorrecta** al problema de la sección crítica de dos procesos, presentada por Hyman en 1966, a fin de competir con el algoritmo de Dekker.

- Muestre un **escenario de ejecución** donde no se cumple la condición de sección crítica.
- ¿**Siempre** funcional mal? Explique.

$Pre : \neg flag_0 \wedge \neg flag_1 \wedge turn = 0$	
$P_0 :$ do $true \rightarrow$ 1 NCS_0 2 ; $flag_0 := true$ 3 ; do $turn = 1 \rightarrow$ 4 do $flag_1 \rightarrow skip$ 5 od 6 ; $turn := 0$ 7 od 8 ; CS_0 9 ; $flag_0 := false$ od	$P_1 :$ do $true \rightarrow$ A NCS_1 B ; $flag_1 := true$ C ; do $turn = 0 \rightarrow$ D do $flag_0 \rightarrow skip$ E od F ; $turn := 1$ G od H ; CS_1 I ; $flag_1 := false$ od

1.4.1 Temas

1.4.2 HowTo

- Buscar camino que haba que ambos programas lleguen a la CS al mismo tiempo
- Analizar caso anterior para determinar que es lo que hace que funcione mal

1.4.3 Resolucion

Ejercicio 4. Debajo se muestra solución **incorrecta** al problema de la sección crítica de dos procesos, presentada por Hyman en 1966, a fin de competir con el algoritmo de Dekker.

- (a) Muestre un **escenario de ejecución** donde no se cumple la condición de sección crítica.
 (b) ¿**Siempre** funcional mal? Explique.

$Pre : \neg flag_0 \wedge \neg flag_1 \wedge turn = 0$	
$P_0 :$ do $true \rightarrow$ 1 NCS_0 2 ; $flag_0 := true$ 3 ; do $turn = 1 \rightarrow$ 4 do $flag_1 \rightarrow skip$ 5 od 6 ; $turn := 0$ 7 od 8 ; CS_0 9 ; $flag_0 := false$ od	$P_1 :$ do $true \rightarrow$ A NCS_1 B ; $flag_1 := true$ C ; do $turn = 0 \rightarrow$ D do $flag_0 \rightarrow skip$ E od F ; $turn := 1$ G od H ; CS_1 I ; $flag_1 := false$ od

⇒

ABCDE

1 2 3 7 8

FGH

b) No, no siempre funciona mal. Solo funciona mal si se hace un cambio de programa dentro del 2do "Do" antes que se modifique turn

1.5 Ej5

Ejercicio 5. El disco rotacional *Seagate Mach.2 Exos 2X14* de 14 TiB e interfaz SAS 3.0, tiene una velocidad de rotacional de 7200 RPM, 4.16 ms de latencia de búsqueda y 524 MiB/s de tasa de transferencia máxima. Este es el disco rotacional más rápido del mundo y eso es gracias a que tiene **dos juegos de cabezales independientes**.

- (a) Indicar cuantos *ms* tarda en dar una vuelta completa.
 (b) Indicar la tasa de transferencia de lectura **al azar** de bloques de 8 MiB ¹.
 (c) Si la tasa de transferencia máxima está dada por la velocidad rotacional que no requiere cambio de pista (no sufre del *seek time*), deducir cuantos MiB almacena cada cilindro.

1.5.1 Temas

- Hard Disk Drives

1.5.2 HowTo

1.5.2.1 A)

1. Pasar RPM a RPS: $RPM/60$
2. Obtener tiempo una vuelta: $1/RPS$

3. Pasar a Ms: $1/\text{RPS} * 1000$

1.5.2.2 B)

IMPORTANTE: dividir $T_{rotation}$ en dos porque hay dos cabezales

T_{seek} = tiempo provisto por fabricante

$$T_{rotation} = \frac{1}{\text{CantRPM}} * 60 * 1000$$

CORRECCION: hay que dividir T_{rot} por 2 otra vez para obtener caso promedio

$$T_{transfer} = \text{TamBloque} / \text{MaxTransfer}$$

$$T_{I/O} = T_{seek} + \frac{T_{rotation}}{2} + T_{transfer}$$

$$R_{I/O} = \text{TamBloque} / T_{I/O}$$

1.5.2.3 C)

$$\text{TamCilindro} = \text{MaxTransfer} * T_{rotation}$$

1.5.3 Resolución

Ejercicio 5. El disco rotacional *Seagate Mach.2 Exos 2X14* de 14 TiB e interfaz SAS 3.0, tiene una velocidad de rotacional de 7200 RPM, 4.16 ms de latencia de búsqueda y 524 MiB/s de tasa de transferencia máxima. Este es el disco rotacional más rápido del mundo y eso es gracias a que tiene dos juegos de cabezales independientes.

- Indicar cuantos ms tarda en dar una vuelta completa.
- Indicar la tasa de transferencia de lectura **al azar** de bloques de 8 MiB ¹.
- Si la tasa de transferencia máxima está dada por la velocidad rotacional que no requiere cambio de pista (no sufre del *seek time*), deducir cuantos MiB almacena cada cilindro.

(*) Dividimos por 2 porque hay dos cabezales

$$a) \text{ vuelta}_{\text{ms}} = \frac{1}{7200 \text{ RPM}} \cdot 60 \cdot 1000 \approx 8,33 \text{ ms}$$

$$b) T_{\text{seek}} = 4,16 \text{ ms} ; T_{\text{rotation}} \stackrel{(*)}{=} \frac{8,33 \text{ ms}}{2} = 4,16 \text{ ms}$$

$$T_{\text{transfer}} = \frac{8 \text{ MiB} \cdot 1000}{524 \text{ MiB/s}} \approx 15,26 \text{ ms}$$

Perdida de ms
↑

$$\begin{aligned} T_{\text{i/o}} &= T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}} \\ &= 4,16 \text{ ms} + 4,16 \text{ ms} + 15,26 \text{ ms} \\ &= 23,58 \text{ ms} \end{aligned}$$

$$R_{\text{i/o}} = \frac{8 \text{ MiB}}{23,58 \text{ ms}} \approx 0,339 \frac{\text{MiB}}{\text{ms}} = 339,27 \frac{\text{MiB}}{\text{s}}$$

∴ La tasa de transferencia para lecturas al azar es de $339,27 \frac{\text{MiB}}{\text{s}}$

$$c) \frac{524 \text{ MiB}}{\text{s}} \cdot 8,33 \text{ ms} = \frac{524 \text{ MiB} \cdot 8,33 \text{ ms}}{1000 \text{ ms}} = 4,36 \text{ MiB}$$

∴ al multiplicar la tasa máxima de transferencia por el tiempo por vuelta, obtuvimos que cada cilindro almacena 4,36 MiB

1.5.4 Correcciones

Para pasar de MiB/ms a MiB/s hay que multiplicar por 1000

$$\frac{1 \text{ MiB}}{15} = \frac{1 \text{ MiB}}{\frac{1}{1000} 5} = \frac{1 \text{ MiB}}{5} \cdot 1000 = 1000 \text{ MiB/s}$$

Trot hay que dividirlo por 2 para tener el caso promedio, y despues por 2 de nuevo

1.6 Ej6 (CORREGIR)

Ejercicio 6. En un sistema de archivos de tipo UNIX, tenemos los bloques de disco dispuestos dentro del *i-nodo* con 12 bloques directos, 1 bloque indirecto, 1 bloque doble indirecto, 1 bloque triple indirecto. Cada bloque es de 4 KiB.

- Calcule la capacidad máxima **de un archivo** para números de bloque de 16, 24 y 32 bits.
- Calcule la capacidad máxima **de la *data region*** para números de bloque de 16, 24 y 32 bits.
- Realice un análisis de que longitud conviene para codificar el número de bloque.

1.6.1 Temas

- File system implementation

1.6.2 HowTo

1.6.2.1 a)

$$indicesPorBloq = \frac{tamBloque}{CantBitsNumBloque}$$

$$tamMax = tamBloque * (cantBloqDir + indicesPorBloq + indPorBloq^2 + indPorBloq^3)$$

1.6.2.2 b)

$$capMaxDR = tamBloque * 2^{CantBitsNumBloque}$$

1.6.2.3 c)

Ver que combinacion de tamaño máximo y capacidad data region permite almacenar más.

Es muy probable que por mas que haya un tamaño máximo mayor, este no pueda ser alocado en la data region. Tener en cuenta esto.

1.6.3 Resolución

1.6.3.1 a)

Ejercicio 6. En un sistema de archivos de tipo UNIX, tenemos los bloques de disco dispuestos dentro del *i-nodo* con 12 bloques directos, 1 bloque indirecto, 1 bloque doble indirecto, 1 bloque triple indirecto. Cada bloque es de 4 KiB.

- (a) Calcule la capacidad máxima **de un archivo** para números de bloque de 16, 24 y 32 bits.
- (b) Calcule la capacidad máxima **de la data region** para números de bloque de 16, 24 y 32 bits.
- (c) Realice un análisis de que longitud conviene para codificar el número de bloque.

a) 32b

$$\text{indicesPorBloque} = \frac{4 \text{ KiB}}{4 \text{ B}} = \frac{2^2 \cdot 2^{10} \cancel{\text{B}}}{2^2 \cancel{\text{B}}} = 2^{10}$$

$$\begin{aligned} \text{tenMax} &= 4 \text{ KiB} \left(12 + 2^{10} + 2^{20} + 2^{30} \right) \\ &= 2^2 \cdot 2^3 \cdot 1074791436 \approx 32 \text{ TiB} = 4 \text{ TiB} \end{aligned}$$

24b

$$\text{indicesPorBloque} = \frac{4 \text{ KiB}}{3 \text{ B}} = \frac{4 \cdot 2^{10} \cancel{\text{B}}}{3 \cancel{\text{B}}} \approx 1365$$

$$\begin{aligned} \text{tenMax} &= 4 \text{ KiB} \left(12 + 1365 + 1365^2 + 1365^3 \right) \\ &= 2^{12} \cdot 3 \cdot (2.545.166.727) = 9,48 \text{ TiB} \end{aligned}$$

16 bits

$$\text{indicesPorBloque} = \frac{4 \text{ KiB}}{2 \text{ B}} = \frac{4 \cdot 2^{10} \cdot \cancel{\text{B}}}{2 \cdot \cancel{\text{B}}} = 2 \cdot 2^{10} = 2^{11} = 2048$$

$$\begin{aligned} \text{tenMax} &= 4 \text{ KiB} \cdot \left(12 + 2^{11} + 2^{11 \cdot 2} + 2^{11 \cdot 3} \right) \\ &= 2^{12} \cdot 3 \cdot (8594130956) \approx 32 \text{ TiB} \end{aligned}$$

1.6.3.2 b)

b) Para calcular la capacidad máxima de la data region podemos usar la sig. fórmula:

$$tamMaxDR = tamBloque \cdot 2^{contBitsNumBlock}$$

Procedamos a hacer los cálculos:

32b:

$$tamMaxDR = 4 \text{ KiB} \cdot 2^{32} = 2^2 \cdot 2^{10} \cdot 8 \cdot 2^{32} = 2^{44} \cdot 2^3 = 2^4 \cdot 2^{40} \cdot 8 = 16 \text{ TiB}$$

24b:

$$tamMaxDR = 4 \text{ KiB} \cdot 2^{24} = 2^2 \cdot 2^{10} \cdot 2^{24} \cdot 8 = 2^6 \cdot 2^{30} \cdot 8 = 64 \text{ GiB}$$

16b:

$$tamMaxDR = 4 \text{ KiB} \cdot 2^{16} = 2^2 \cdot 2^{10} \cdot 2^{16} \cdot 8 = 2^8 \cdot 2^{20} \cdot 8 = 256 \text{ MiB}$$

c)

Resumen

	capMaxArchivo	capMaxDR
32b	4 TiB	16 TiB
24b	9 TiB	64 GiB
16b	32 TiB	256 MiB

Teniendo en cuenta que nos interesa maximizar la data region y a la vez mantener un tamaño aceptable para los archivos, llegamos a la conclusión de que la mejor opción es utilizar num. de bloque de 32b.

Además, que para el caso de los 32b podemos meter hasta 4 archivos de tamaño máximo en la data region, lo cual es imposible en los demás casos

1.6.4 Correcciones

Prestar atención en no poner resultado en TeraBits en vez de terabytes

2 F2

2.0.1 Ej1

Ejercicio 1. Explique detalladamente como funcionan estos programas indicando cuantos procesos se crean, que hacen los padres, etc.

Suponga que en ambos casos se invocan con

```
$ ./a.out ./a.out ./a.out ./a.out ./a.out ./a.out ./a.out
```

```
int main(int argc, char ** argv) {
    char buf[L] = {'\0'};
    if (argc<=1)
        return 0;
    int rc = fork();
    if (rc<0)
        return -1;
    if (0==rc) {
        close(2);
        open(argv[0], 0);
        read(2, buf, L);
        write(1, buf, L);
    } else {
        argv[argc-1] = NULL;
        execvp(argv[0], argv);
    }
}
```

```
int main(int argc, char ** argv) {
    if (argc<=1)
        return 0;
    int rc = fork();
    if (rc<0)
        return -1;
    else if (0==rc)
        main(argc-2, argv)
    else {
        argv[argc-1] = NULL;
        argv++;
        execvp(argv[0], argv);
    }
}
```

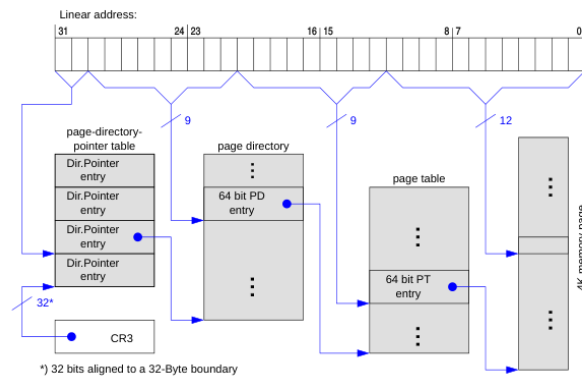
2.0.2 Temas

2.0.3 HowTo

2.0.4 Resolucion

2.1 Ej2

Ejercicio 2. Para el sistema de paginado i386 con PAE de (2, 9, 9, 12) que transforma de 32 bits virtual a 36 bits físicos. Los marcos físicos ahora pasan de 20 bits (5 dígitos hexa) a 24 bits (6 dígitos hexa).



Sabiendo que CR3=0x20000 y que el contenido de los marcos físicos son los siguientes:

0x000000	0x010000	0x020000
-----	-----	-----
0x000: 0x010000 (P)	0x000: 0x000000 (P)	0x000: 0x000000 (P)
0x001: 0x010000 (P)	0x001: 0x000001 (P)	0x001: 0x000000 (P)
...	...	0x002: 0x020000 (P)
...	...	0x003: 0x000000 (P)
0x1FE: 0x010000 (P)	0x1FE: 0x0001FE (P)	
0x1FF: 0x000000 (P)	0x1FF: 0x0001FF (P)	

- (a) Pasar de virtual a física: 0x00000EEF, 0x00001FEE, 0x00200EEF, 0x00201FEE.
- (b) Pasar de virtual a física 0x80000CFF, 0x8001F000
- (c) Pasar de física a todas las virtuales 0x000000FFF, 0x020000FFF, 0x030000FFF

2.1.1 Temas

2.1.2 HowTo

2.1.3 Resolucion

2.2 Ej3

Ejercicio 3.

- (a) Muestre un escenario de ejecución que termine en $a=\{0,1,0,1,0,1 \dots\}$ para la Figura 1, suponga siempre **atomicidad es línea-a-línea**.
- (b) Agregue sincronización con semáforos para que el valor de salida sea siempre el del punto anterior. Puede colocar condicionales (if) sobre el valor de las variables i y j para hacer wait/post de los semáforos. No puede tocar los incrementos de las variables y las asignaciones al arreglo.

Pre: $0 < N \wedge i, j = 0, 0 \wedge (\forall k : 0 \leq k < N : a[k] = 2)$	
<pre> 1 P0: while (i<N) { 2 a[i] = 0; 3 ++i; }</pre>	<pre> a P1: while (j<N) { b a[j] = 1; c ++j; }</pre>

Figura 1: Multiprograma

2.2.1 Temas

2.2.2 HowTo

2.2.3 Resolucion

Ejercicio 3.

- (a) Muestre un escenario de ejecución que termine en $a=\{0,1,0,1,0,1 \dots\}$ para la Figura 1, suponga siempre **atomicidad es línea-a-línea**.
- (b) Agregue sincronización con semáforos para que el valor de salida sea siempre el del punto anterior. Puede colocar condicionales (if) sobre el valor de las variables i y j para hacer wait/post de los semáforos. No puede tocar los incrementos de las variables y las asignaciones al arreglo.

Pre: $0 < N \wedge i, j = 0, 0 \wedge (\forall k: 0 \leq k < N: a[k] = 2)$	
<pre> 1 P0: while (i < N) { 2 a[i] = 0; 3 ++i; }</pre>	<pre> a P1: while (j < N) { b a[j] = 1; c ++j; }</pre>

Figura 1: Multiprograma

bc 23 23 bc bc 23 23 bc ...
 $1 \rightarrow 0, 0 \rightarrow 1, 1 \rightarrow 0, 0 \rightarrow 1$
 $\Rightarrow a = \{ \underline{0}, \underline{1}, \underline{0}, \underline{1}, \dots \}$

```

sem_t p0, p1;

sem_init(&p0, 0, 0);
sem_init(&p1, 0, 1);

while (i < N) {
    sem_wait(&p0);
    a[i] = 0;
    ++i;
    if (i % 2 == 0) {
        sem_post(&p1);
        sem_post(&p1);
    }
}

while (j < N) {
    sem_wait(&p1);
    a[j] = 1;
    ++j;
    if (j % 2 == 1) {
        sem_post(&p0);
        sem_post(&p0);
    }
}
```

```

sem_t p0, p1;

sem_init(&p0, 0, 0);
sem_init(&p1, 0, 1);

while (i < N) {
    sem_wait(&p0);
    a[i] = 0;
    ++i;
    if (i % 2 == 0) {
        sem_post(&p1);
        sem_post(&p1);
    }
}

while (j < N) {
    sem_wait(&p1);
    a[j] = 1;
    ++j;
    if (j % 2 == 1) {
        sem_post(&p0);
        sem_post(&p0);
    }
}
```

2.3 Ej4

Ejercicio 4. Considere los procesos P0 y P1 a continuación, donde las sentencias **no son** atómicas.

Pre: $n = 0$	
P0 : while ($n < 100$) { $n = n * 2$; }	P1 : while ($n < 100$) { $n = n + 1$; }

- ¿Qué valores posibles puede tomar n al terminar el multiprograma?. Explique.
- Sincronizar con semáforos para que se comporte como la versión atómica dada en el práctico que da valores de n entre 100 y 200.

2.3.1 Temas

2.3.2 HowTo

2.3.3 Resolucion

Ejercicio 4. Considere los procesos P0 y P1 a continuación, donde las sentencias **no son** atómicas.

Pre: $n = 0$	
P0 : while ($n < 100$) { $n = n * 2$; }	P1 : while ($n < 100$) { $n = n + 1$; }

- ¿Qué valores posibles puede tomar n al terminar el multiprograma?. Explique.
- Sincronizar con semáforos para que se comporte como la versión atómica dada en el práctico que da valores de n entre 100 y 200.

a) Valor máximo: $\overset{\text{P1 bucle}}{100} \cdot \underset{\text{P0}}{2} = 200$

Valor mínimo: $\underset{\text{Solo P1}}{100}$

Como n puede tomar la forma tanto de 2^n como de $2^n + 1$ (números pares e impares) y su valor máximo y mínimo es 200 y 100, tenemos que n puede tomar cualquier valor entero entre 100 y 200.

b) Se comporta igual que la versión atómica

2.4 Ej5

Ejercicio 5. Usted forma parte de un equipo al que le fue encargado el diseño de un sistema de archivos *UNIX-like* y uno de sus compañeros propuso la idea de aprovechar los 75 bytes que sobran en los i-nodos (Cada i-nodo ocupa 181 bytes y se decidió “alinearlo” a 256 bytes para que entraran 2 por bloque de disco) almacenando allí los archivos que no superen esta capacidad¹.

- (a) **Discuta** porque si o porque no esta **decisión de diseño** es acertada, en cuanto a eficiencia, velocidad y simpleza. Soporte sus argumentos con las mediciones² de la Figura 2.
- (b) ¿Tiene sentido que estos 75 bytes **siempre se utilicen** sin importar la longitud del archivo? Discuta.

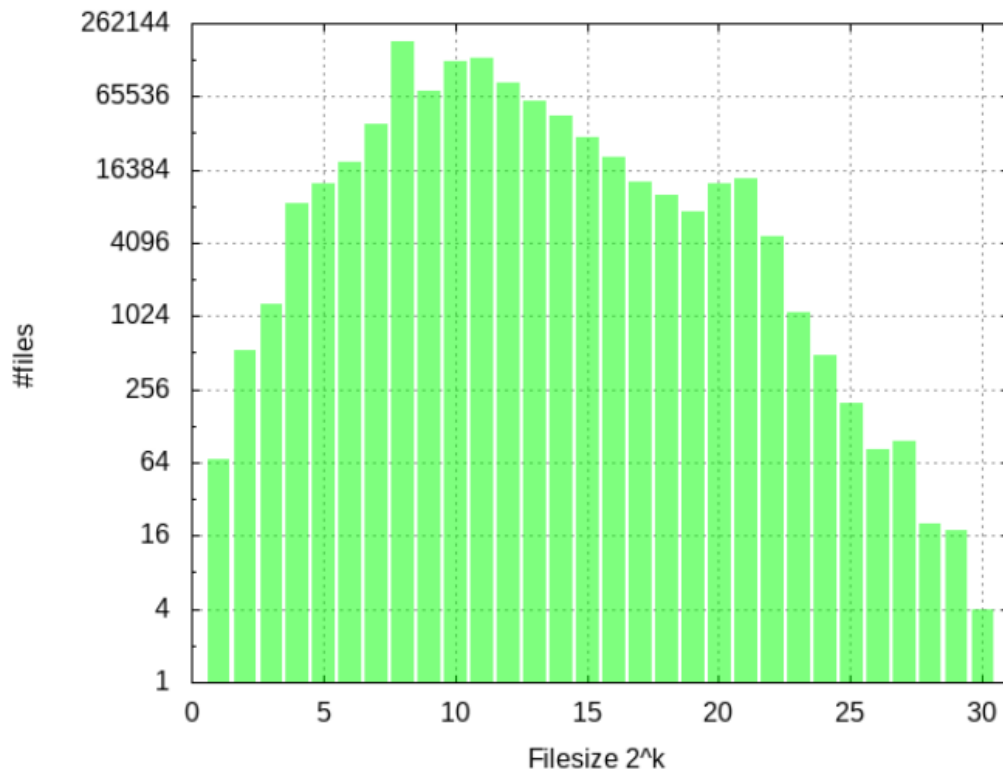


Figura 2:

2.4.1 Temas

2.4.2 HowTo

2.4.3 Resolucion

2.4.3.1 a)

Esta decision de diseño posee ciertas ventajas y desventajas.

En cuanto a la velocidad, como uno de los mayores cuellos de botella es la velocidad del disco. Si los metadatos se guardan en el mismo bloque que se guardan los datos, podemos obtener tanto los metadatos como los datos del archivo en una sola lectura. Por lo cual, podemos determinar que esta decision de diseño sería rápida y eficiente. Aunque quizás se requiera que se utilice un mayor número de instrucciones para acceder a los 75 bytes de datos. Uno de los contras de esta idea es que se incrementa la complejidad de los datos, y como antes mencionamos, seguramente aumenta la cantidad de instrucciones que hace falta ejecutar para poder acceder a los datos.

Teniendo todo esto en cuenta, como en la figura podemos observar que hay una gran cantidad de archivos con tamaño menor a 2^{10} creo que se podría conseguir un aumento en la velocidad significativo al guardar los datos en el mismo bloque que el inodo.

2.4.3.2 b)

Creo que no tendría sentido que los 75 bytes se usen si el tamaño del archivo supera esta cantidad. Esto es debido a que se pierde en gran medida la localidad de los datos. Ya que el inodo puede estar bastante alejado de donde se encuentran los datos.

Aunque también se puede utilizar el argumento de que de todas maneras la información del inodo será leída. Por lo cual, daría igual si almacenamos información en ese mismo bloque.

2.4.4 Correcciones

(Sacado de clase consulta Wolo)

En el A no hay mucha desventaja que digamos.

La desventaja del punto B es que la mayoría de programas leen en bloques. Entonces tenés que ir calculando $\text{tamBloque} - 75B$ y así Después tenes que calcular el segundo bloque, y así. Esto complejiza las cosas.

Los programas que leen de a bloques van a andar un poco peor

2.5 Ej6

Ejercicio 6. En un sistema de archivos de tipo UNIX, tenemos los bloques de disco dispuestos dentro del *i-nodo* con 12 bloques directos, 1 bloque indirecto y 1 bloque doble indirecto. Cada bloque es de 4 KiB.

- (a) Calcule la capacidad máxima del *block pool* para números de bloque de 16, 24 y 32 bits.
 - (b) Calcule la capacidad máxima de un archivo para números de bloque de 16, 24 y 32 bits.
 - (c) Realice un análisis de que longitud de número de bloque conviene.
-

2.5.1 Temas

2.5.2 HowTo

Data region = block pool

2.5.3 Resolucion

2.5.3.1 a) y b)

Ejercicio 6. En un sistema de archivos de tipo UNIX, tenemos los bloques de disco dispuestos dentro del *i-nodo* con 12 bloques directos, 1 bloque indirecto y 1 bloque doble indirecto. Cada bloque es de 4 KiB.

(a) Calcule la capacidad máxima del *block pool* para números de bloque de 16, 24 y 32 bits.

(b) Calcule la capacidad máxima de un archivo para números de bloque de 16, 24 y 32 bits.

(c) Realice un análisis de que longitud de número de bloque conviene.

a) Para calcular capacidad max block pool debemos usar la siguiente formula:

$$capMaxBP = blockSize \cdot 2^{cantBitsNumBloque}$$

$$16b: capMaxBP = 4KiB \cdot 2^{16} = 2^2 \cdot 2^{10} \cdot 2^{16} B = 2^8 \cdot 2^{20} B = 256 MiB$$

$$24b: capMaxBP = 4KiB \cdot 2^{24} = 2^2 \cdot 2^{10} \cdot 2^{24} B = 2^6 \cdot 2^{30} B = 64 GiB$$

$$32b: capMaxBP = 4KiB \cdot 2^{32} = 2^2 \cdot 2^{10} \cdot 2^{32} B = 2^4 \cdot 2^{40} B = 16 TiB$$

b) Para calcular la cap. maxima de un archivo debemos usar la sig. formula:

$$tanMax = blockSize \cdot (cantBloqDir + indPrBloq + indPrBloq^2)$$

$$con \quad indPrBloq = \frac{blockSize}{cantBitsNumBloq}$$

$$16b: indPrBloq = \frac{4KiB}{16} = \frac{2^2 \cdot 2^{10} B}{2^4} = 2 \cdot 2^{10} = 2^{11}$$

$$tanMax = 4KiB \cdot (12 \cdot 2^{11} \cdot 2^{11 \cdot 2}) = 2^2 \cdot 2^{10} B \cdot (12 \cdot 2^{11} \cdot 2^{22})$$

$$= 17\,188\,306\,944\,B = 16\,GiB$$

$$24b: indPrBloq = \frac{4KiB}{3B} = \frac{2^2 \cdot 2^{10} B}{3B} = \frac{2^{12}}{3} = 1365$$

$$tanMax = 4KiB \cdot (12 \cdot 1365 \cdot 1365^2) = 2^{12} B \cdot (12 \cdot 1365 \cdot 1365^2)$$

$$= 7\,677\,409\,792\,B = 7,11\,GiB$$

$$32b: indPrBloq = \frac{4KiB}{4B} = \frac{2^2 \cdot 2^{10} B}{2^2 B} = 2^{10} =$$

$$tanMax = 4KiB \cdot (12 \cdot 2^{10} \cdot 2^{20}) = 2^{12} B \cdot (12 \cdot 2^{10} \cdot 2^{20})$$

$$= 4\,299\,210\,752\,B = 4\,GiB$$

2.5.3.2 c) Conviene un número de bloque de 32b ya que es el que brinda mayor capacidad a la block pool y ademas permite que el archivo de mayor tamaño posible llegue a entrar hasta 4 veces en la block pool.

3 F3

3.1 Ej1

Ejercicio 1. Explique detalladamente como funcionan estos programas. Suponga que en ambos casos se invocan con

```
$ ./a.out ./a.out ./a.out ./a.out ./a.out ./a.out ./a.out
```

```
int main(int argc, char ** argv) {
    argc--;
    if (0<argc) {
        argv++;
        execvp(argv[0], argv);
    }

    return 0;
}
```

```
int main(int argc, char ** argv) {
    if (argc<=1)
        return 0;
    int rc = fork();
    argv[argc-1] = NULL;
    --argc;
    if (rc<0)
        return -1;
    else if (0==rc)
        main(argc, argv);
    else
        execvp(argv[0], argv);
}
```

3.1.1 Temas

3.1.2 HowTo

3.1.3 Resolucion

3.2 Ej2

Ejercicio 2. Mostrar la secuencia de accesos a la memoria física que se produce al ejecutar este programa assembler x86_32, donde $Base_{code} = 0x1000$, $Bounds_{code} = 0x100$, $Base_{data} = 0x2000$, $Bounds_{data} = 0x10$

La versión original de este código salió en el número de abril de 1967 de Cosmopolitan.

```
0: mov    $0x0,%edx
6: mov    0x0(%edx),%ecx
12: mov    0x4(%edx),%eax
18: add    $0x8,%edx
21: add    (%edx),%ecx
23: add    $0x4,%edx
26: sub    $0x1,%eax
29: jne     21
```

```
0x0: .long 0x0
0x4: .long 0x4
0x8: .long 0x3
0xC: .long 0x7
0x10: .long 0x2
0x14: .long 0x1
0x18: .long 0x5
```

3.2.1 Temas

3.2.2 HowTo

- Si el código accede a una dir mayor o igual a $base_{code} + bounds_{code}$ da SegFault
- Si se accede a un dato mayor o igual a $base_{data} + bounds_{data}$ da SegFault
- `mov 0x4(%edx), %eax` means that you offset %edx by 4, and you move the content of that direction into eax
- `$` identifies immediate values
- Si no hay `$` probablemente sea un acceso a memoria
- `jne` jump if not empty (not zero). Reads flag from previous operation

Correccion: Las direcciones deben ser menor estricto que $base + bound$

3.2.3 Resolucion

$base_{code} = 0x1000$

$bounds_{code} = 0x100$

$base_{data} = 0x2000$

$bounds_{data} = 0x10$

linea	registros	virtual	fisica	SF?
0	edx=0x0	0x0	0x1000	
6	ecx=0x0	0x6, 0x0	0x1006, 0x2000	
12	eax=0x4	0xC, 0x4	0x100C, 0x2004	
18	edx=0x8	0x12	0x1012	
21	ecx=0x3	0x15, 0x8	0x1015, 0x2008	
23	edx=0xC	0x17	0x1017	
26	eax=0x3	0x1A	0x101A	
21	ecx=0x3+0x7=0xA	0x15, 0xC	0x1015, 0x200C	
23	edx=0x4+0xC = 0x10	0x17	0x1017	
26	eax=0x2	0x1A	0x101A	
21	ecx=0xA+0x2=0xC	0x15, 0x10	0x1015, 0x2010	SF!

Así seguiría el programa (lo dejo para que quede registro):

23	edx=0x14	0x17	0x1017	
26	eax=0x1	0x1A	0x101A	
21	ecx=0xC+0x1=0xD	0x15, 0x14	0x1015, 0x2014	SF!
23	edx=0x18	0x17	0x1017	
26	eax=0x0	0x1A	0x101A	

3.2.4 Correccion

Las direcciones deben ser menor estricto que base + bounds

3.3 Ej3

Ejercicio 3. Para el sistema de paginado i386 (10, 10, 12), sabiendo que CR3=0x0C01A y que el contenido de los marcos físicos son los siguientes:

0x0C0CA	0x0C01A
-----	-----
0x000: 0x0C01A (P)	0x000: 0x0C0CA (P)
0x001: 0x0C0CA (P)	0x001: 0x0C01A (P)
...	...
...	...
0x3FE: 0x0C01A (P)	0x3FE: 0x0C0CA (P)
0x3FF: 0x0C0CA (P)	0x3FF: 0x0C01A (P)

- (a) Pasar de virtual a física: 0x00000BAD, 0x00001BE1, 0x00400BA1, 0x00401BE5.
- (b) Pasar de física a **todas** las virtuales: 0x0C0CABAD, 0x0C01ABE5, 0xDEADBEEF, 0xDABBAD00.

3.3.1 Temas

3.3.2 HowTo

3.3.3 Resolucion

3.4 Ej4

Ejercicio 4. Considere el siguiente multiprograma de 3 componentes, donde x es compartida y el **incremento NO es atómico**. Exprese de manera concisa todos resultados posibles de la variable compartida x para cada uno de estos valores iniciales de semáforo $s=\{0,1,2,3\}$

Pre: $x=0 \wedge s=?$		
P0 : <code>down(s);</code> <code>x = x+1;</code> <code>up(s);</code>	P1 : <code>down(s);</code> <code>x = x+1;</code> <code>up(s);</code>	P2 : <code>down(s);</code> <code>x = x+1;</code> <code>up(s);</code>
Post: $x=?$		

3.4.1 Temas

3.4.2 HowTo

3.4.3 Resolucion

Ejercicio 4. Considere el siguiente multiprograma de 3 componentes, donde x es compartida y el **incremento NO es atómico**. Exprese de manera concisa todos resultados posibles de la variable compartida x para cada uno de estos valores iniciales de semáforo $s=\{0,1,2,3\}$

Pre: $x=0 \wedge s=?$		
P0 : down(s); $x = x+1$; up(s);	P1 : down(s); $x = x+1$; up(s);	P2 : down(s); $x = x+1$; up(s);
Post: $x=?$		

$s=0$:

Se produce un deadlock y ningun programa puede avanzar porque se quedan trabados esperando a que el valor de s sea distinto a 0.

El programa nunca termina y $x=0$.

$s=1$:

$x = 3$.

Cada programa se va turnando para sumar +1 a x

$s=2$:

$x \in \{2, 3\}$

Puede darse que el incremento sea exactamente al mismo tiempo. Por ejemplo, supongamos que se ejecutan P0 y P1 a la vez, ambas suman 1, pero se da el caso de que lo hacen al mismo tiempo entonces x termina con valor igual a 1 en vez de 2.

$s=3$:

$x \in \{1,2,3\}$

Similar a lo que sucede anteriormente, puede que los 3 programas se ejecuten exactamente al mismo tiempo, causando que el valor que se guarde sea 1. De la misma manera puede pasar esto con dos programas o con ninguno, haciendo que los posibles resultados para x sean 1, 2 y 3.

3.5 Ej5

Ejercicio 5. Para el programa de la Figura 1, donde la **atomicidad es línea-a-línea**:

- Muestre un (1) escenario de ejecución con $N = 5$ de forma tal que la salida del multiprograma cumpla con $a = [0, 1, 0, 1, 0]$.
- Muestre un (1) escenario de ejecución con $N = 4$ de forma tal que la salida del multiprograma cumpla con $a = [0, 0, 2, 2]$.
- Expresé **todos los valores posibles** de salida de arreglo a para la Figura 2. Explique.

Pre: $0 < N \wedge i, j = 0 \wedge (\forall k : 0 \leq k < N : a[k] = 2)$	
<pre> 1 P0: while (i < N) { 2 a[i] = 0; 3 ++i; }</pre>	<pre> a P1: while (j < N) { b a[j] = 1; c ++j; }</pre>
a=?	

Figura 1: Concurrent Vector Writing (CVW)

Pre: $0 < N \wedge i, j = 0 \wedge s, t = 0, 1 \wedge (\forall k : 0 \leq k < N : a[k] = 2)$	
<pre> P0: while (i < N) { sem_wait(s); a[i] = 0; ++i; sem_post(t); }</pre>	<pre> P1: while (j < N) { sem_wait(t); a[j] = 1; ++j; sem_post(s); }</pre>
a=?	

Figura 2: CVW sincronizado

3.5.1 Temas

3.5.2 HowTo

3.5.3 Resolucion

Ejercicio 5. Para el programa de la Figura 1, donde la **atomicidad es línea-a-línea**:

- Muestre un (1) escenario de ejecución con $N = 5$ de forma tal que la salida del multiprograma cumpla con $a = [0, 1, 0, 1, 0]$.
- Muestre un (1) escenario de ejecución con $N = 4$ de forma tal que la salida del multiprograma cumpla con $a = [0, 0, 2, 2]$.
- Expresé **todos los valores posibles** de salida de arreglo a para la Figura 2. Explique.

Pre: $0 < N \wedge i, j = 0 \wedge (\forall k : 0 \leq k < N : a[k] = 2)$	
1 P0: while (i < N) {	a P1: while (j < N) {
2 a[i] = 0;	b a[j] = 1;
3 ++i;	c ++j;
}	}
a = ?	

Figura 1: Concurrent Vector Writing (CVW)

Pre: $0 < N \wedge i, j = 0 \wedge s, t = 0, 1 \wedge (\forall k : 0 \leq k < N : a[k] = 2)$	
P0: while (i < N) {	P1: while (j < N) {
sem_wait(s);	sem_wait(t);
a[i] = 0;	a[j] = 1;
++i;	++j;
sem_post(t);	sem_post(s);
}	}
a = ?	

Figura 2: CVW sincronizado

a) abc123 123abc abc123 123abc abc123

b) Se puede comenzar con abc123 abc123, pero luego no hay forma de hacer que los últimos dos elementos de a sean 2 con $N=4$, ya que si P0 o P1 hacen más de dos iteraciones cada uno, se terminaría sobrescribiendo los valores ya presentes en a .

c) El único valor posible es que todos los valores de a sean iguales a 0, ya que siempre P1 escribe un 1 y después P0 sobrescribe este 1 con un 0. Esto se debe a que P1 siempre se ejecuta antes que P0, ya que el semáforo de P1 arranca en 1, mientras que el de P0 arranca en 0.

3.6 Ej6

Ejercicio 6. El disco *Western Digital Red NAS* de 4 TiB e interfaz SATA tiene una velocidad de rotacional de 5400 RPM, 5.6ms de latencia de búsqueda y 150 MiB/s de tasa de transferencia máxima.

- Indicar cuantos *ms* tarda en dar una vuelta completa.
- Indicar la tasa de transferencia de lectura **al azar** de bloques de 16 MiB.
- ¿La tasa de transferencia al azar de bloques de 128 MiB será mayor o menor a la anterior? Calcule y/o explique.
- Si la tasa de transferencia máxima está dada por la velocidad rotacional que no requiere cambio de pista (no sufre del *seek time*), deducir cuantos MiB almacena cada pista.

3.6.1 Temas

3.6.2 HowTo

3.6.3 Resolución

Ejercicio 6. El disco *Western Digital Red NAS* de 4 TiB e interfaz SATA tiene una velocidad de rotacional de 5400 RPM, 5,6ms de latencia de búsqueda y 150 MiB/s de tasa de transferencia máxima.

- Indicar cuantos ms tarda en dar una vuelta completa.
- Indicar la tasa de transferencia de lectura **al azar** de bloques de 16 MiB.
- ¿La tasa de transferencia al azar de bloques de 128 MiB será mayor o menor a la anterior? Calcule y/o explique.
- Si la tasa de transferencia máxima está dada por la velocidad rotacional que no requiere cambio de pista (no sufre del *seek time*), deducir cuantos MiB almacena cada pista.

a) ^{obtener tiempo vuelta}
Dividimos 1 por los RPM, multiplicamos por 60 para obtener segundos, multiplicamos por 1000 para obtener ms.

$$TiempoVuelta = \frac{1}{5400} \cdot 60 \cdot 1000 = 11,11 \text{ ms}$$

(*) dividido para obtener resultado promedio

$$b) T_{seek} = 5,6 \text{ ms} ; T_{rot} = \frac{11,11 \text{ ms}}{2} = 5,55 \text{ ms} ;$$

$$T_{transfer} = \frac{16 \text{ MiB}}{150 \frac{\text{MiB}}{\text{s}}} = \frac{16 \text{ s}}{150} = \frac{16 \cdot 1000 \text{ ms}}{150} = 106,66 \text{ ms}$$

$$T_{i/o} = 5,6 \text{ ms} + 5,55 \text{ ms} + 106,66 \text{ ms} \\ = 117,81 \text{ ms}$$

$$R_{i/o} = \frac{16 \text{ MiB}}{117,81 \text{ ms}} = \frac{16 \text{ MiB}}{\frac{117,81 \text{ s}}{1000}} = \frac{16000 \text{ MiB}}{117,81 \text{ s}} = 135,81 \text{ MiB/s}$$

c) Para determinar si es mayor o menor calculamos $T_{transfer}$ y comparemos

$$T_{transfer} = \frac{128 \text{ MiB}}{150 \frac{\text{MiB}}{\text{s}}} = \frac{128 \text{ s}}{150} = \frac{128 \cdot 1000 \text{ ms}}{150} = 853 \text{ ms}$$

Por ende, como $T_{transfer}$ es mucho mayor que el otro caso, y teniendo en cuenta que T_{seek} y T_{rot} no se ven afectados por un cambio de tamaño de bloque, concluir que la tasa de transferencia al azar de bloques de 128 MiB será menor a la anterior.

$$d) \text{ Tamaño Cilindro} = \text{MaxTransfer} \cdot \text{TiempoVuelta} = 150 \frac{\text{MiB}}{\text{s}} \cdot 11,11 \text{ ms} \\ = \frac{150 \text{ MiB}}{1000 \text{ ms}} \cdot 11,11 \text{ ms} = 1,66 \text{ MiB}$$

3.7 Ej7

Ejercicio 7. En un sistema de archivos de tipo UNIX, tenemos los bloques de disco dispuestos dentro del *i-nodo* con 12 bloques directos, 1 bloque indirecto y 1 bloque doble indirecto. Cada bloque es de 4 KiB y los números de bloque ocupan 24 bits.

- (a) Calcule la capacidad máxima de la *data region* del sistema de archivos.
- (b) Calcule la capacidad máxima de un archivo.
- (c) Calcule para que tamaño se pasa de bloques directos a indirecto y para que tamaño de indirecto a doble indirecto.

3.7.1 Temas

3.7.2 HowTo

3.7.3 Resolucion

Ejercicio 7. En un sistema de archivos de tipo UNIX, tenemos los bloques de disco dispuestos dentro del *i-nodo* con 12 bloques directos, 1 bloque indirecto y 1 bloque doble indirecto. Cada bloque es de 4 KiB y los números de bloque ocupan 24 bits.

- (a) Calcule la capacidad máxima de la *data region* del sistema de archivos.
- (b) Calcule la capacidad máxima de un archivo.
- (c) Calcule para que tamaño se pasa de bloques directos a indirecto y para que tamaño de indirecto a doble indirecto.

$$a) \quad 4\text{KiB} \cdot 2^{24} = 2^{12}\text{B} \cdot 2^{24} = 2^6 \cdot 2^{30}\text{B} = 64\text{GiB}$$

$$b) \quad \text{indPorBlock} = \frac{4\text{KiB}}{3\text{B}} = \frac{2^{12}\cancel{\text{B}}}{3\cancel{\text{B}}} = 1365$$

$$\begin{aligned} \text{capMaxFile} &= 4\text{KiB} \cdot (12 + 1365 + 1365^2) \\ &= 2^{12}\text{B} \cdot (1864602) = 7,11\text{GiB} \end{aligned}$$

c) Directos:

$$\text{MaxSize} = 4\text{KiB} \cdot 12 = 2^{12}\text{B} \cdot 12 = 49.152\text{B} = 48\text{KiB}$$

Indirectos:

$$\text{MaxSize} = 4\text{KiB} \cdot (12 + 1365) = 5.640.192\text{B} = 5,37\text{MiB}$$

Conclusion

$$0 < \text{fileSize} < 48\text{KiB} \Rightarrow \text{use bloques directos}$$

$$48\text{KiB} < \text{fileSize} < 5,37\text{MiB} \Rightarrow \text{use bloques indirectos}$$

$$5,37\text{MiB} < \text{fileSize} < \underbrace{7,11\text{GiB}}_{\substack{\text{Tamaño} \\ \text{máximo}}} \Rightarrow \text{use bloques doble indirectos}$$

4 F4

4.1 Ejercicio 1:

un código que se ejecutaba como `./a.out ./a.out ./a.out ./a.out ./a.out ./a.out`, forkeaba el y el hijo hacía: `close(1)` y después `read(1, buf,L)` //buf era un `char[L]`

Después `close(0)` y `write(0,buf,L)` creo

El padre hacía `execvp(argv[1], argv+1)`

4.2 Ejercicio 2

traducir memoria virtual a física en paginación de 3 niveles, después te daban una física y tenías que dar todas las virtuales que apuntaban a esa

4.3 Ejercicio 3

un loop de sumas en un arreglo en risc-v, tenías que dar la traza de memoria física (traducir direcciones virtuales de instrucciones y algún fetch data que pasaba por ahí) en un punto un elemento del arreglo no tenía una traducción válida

4.4 Ejercicio 4

semaforos, 2 loops, 1 pone los elementos del arreglo en 0, el otro en 1, tenías que sincronizarlo para que el arreglo final quede $\{1,0,1,\dots,1\}$

4.5 Ejercicio 5

tenías que poner read/write secuencialmente en cada una de las estructuras correspondientes a la hora de ejecutar algún comando relacionado con el filesystem

El primero era `create /foo/bar` y después 2 `write()`;

5 F5 (final Eze)

5.1 Ej3

5.1.1 Howto

En el a los programas son independientes.

5.2 Ej4

5.2.1 Howto

Cuando se hace read/write del inodo de bar, es para ver si ya existe un archivo bar con proteccion de escritura por ejemplo

Osea, primero lo lees para ver si lo podes escribir y recien ahí lo escribis

O puede que justo otro programa haya escrito el inodo

5.3 Ej5

5.3.1 Howto

Hay que basarse solo en los inodos alocados

Si los bloques de un inodo apuntan todos a cero, probablemente sea un archivo sin tamaño que se alocó todo por un calloc o similar. Que en vez de escribir todo a memoria se hace que apunte todo a un mismo bloque que tiene todo ceros.

Si un archivo lo llenas de ceros, se apuntan todos los bloques a un mismo bloque lleno de ceros. Es muy similar a paginacion

5.4 Ej tabla create read/writes

5.4.1 Howto

5.4.1.1 Creacion

1. Leer inodo root
2. Leer data root, encontrar foo.
3. Leer inodo foo.
4. Leer data foo
5. Alocar inodo
6. Actualizar data foo
7. Leer inodo bar (ver que no haya sido escrito por otro programa) y escribir bar.
8. Actualizar fecha ultimo acceso foo

5.4.1.2 Escritura

1. Lee inodo archivo
2. Aloca data block en d-bitmap
3. Escribe datos en bloque
4. Actualiza metadatos en inodo archivo (tamaño, modify time, etc)

5.5 Ej tabla open read/writes (no tomado, pero lo anoto por las dudas)

5.5.1 Howto

5.5.1.1 Apertura

1. Leer inodo root
2. Leer data root, encontrar foo.
3. Leer inodo foo.
4. Leer data foo
5. Leer inodo bar

5.5.1.2 Lectura

1. Lee el inodo de bar
2. Lee el bloque correspondiente
3. Actualiza tiempo de acceso, offset en el inodo bar

6 Final 7-12-2023

6.1 Ej6

6.1.1 Howto

6.1.1.1 Calcular tamaño i-bmap

$$ibmapSize = \frac{inodeAmount}{2^3}$$

Se divide cantidad de inodos por 2^3 para pasarlo a bytes.

Pasar a una unidad más conveniente

6.1.1.2 Calcular tamaño d-bmap

$$dbmapSize = \frac{2^{indexSize}}{2^3}$$

Se divide cantidad de bloques por 2^3 para pasarlo a bytes.

Despues se lo pasa a GiB o MiB segun convenga

6.1.1.3 Calcular tamaño inode-table

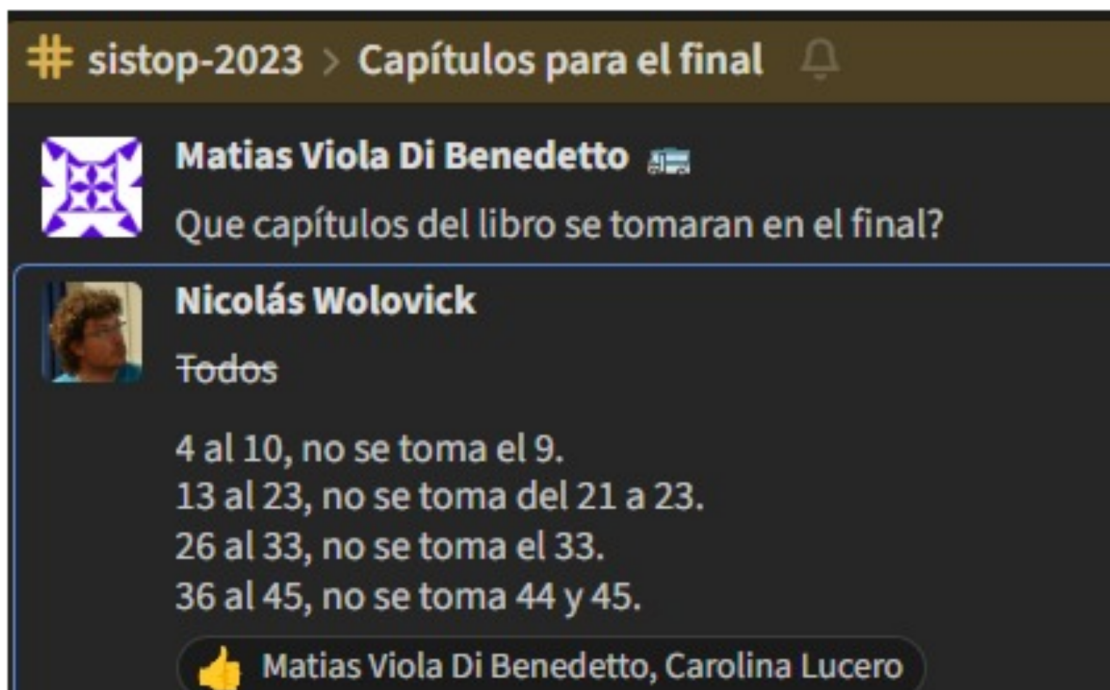
$$itableSize = inodeSize * inodeAmount$$

6.1.1.4 Calcular tamaño data-region

$$DRSize = 2^{indexSize} * blockSize$$

7 Extra

7.1 Caps final



7.2 Notas

Prestar atencion nombres main()

revisar uso fflush()

7.2.1 ¿Qué es máxima concurrencia?

La definición sería que es posible una ejecución correcta, según la definición de correcta para este problema en particular, donde se da la mayor cantidad de interleavings posibles. Un solo interleaving, que es así, concurrencia mala. Dos interleavings, o sea, así y así, está bueno, porque en realidad puede ser un procesador que termine primero y el otro después, entonces está bueno eso.